Laboratorio 7 — NLP end-to-end y Embeddings

CC3092 - Deep Learning y Sistemas Inteligentes

1. Datos e investigacion

Datos. WikiText-103 (crudo, Wikipedia) como corpus de entrenamiento, AG News (4 clases) como tarea de clasificacion, y glove-wiki-gigaword-100 como referencia preentrenada (Wikipedia+Gigaword, 6B tokens, mismo dominio que nuestro corpus). Benchmarks: questions-words.txt (19,544 analogias, 14 categorias), WordSim-353 (seleccion de hiperparametros) y SimLex-999 (reservado para evaluacion final).

nn.Embedding guarda una tabla (num_embeddings, embedding_dim) indexada por id entero; padding_idx fija un vector (tipicamente cero, sin gradiente) para el token de relleno; sparse=True usa gradientes dispersos, mas eficiente con vocabularios grandes y optimizadores como SparseAdam. from_pretrained(vectors, freeze=True/False) crea la tabla a partir de vectores ya entrenados (p.ej. GloVe); freeze controla si se actualizan durante el entrenamiento posterior. nn.EmbeddingBag combina lookup + agregacion (mode='mean'|'sum'|'max') en una sola operacion eficiente: recibe un tensor flat de ids concatenados mas un tensor offsets que marca donde empieza cada bolsa (documento/oracion), evitando materializar padding; es el bloque natural para clasificacion de texto con “bag of embeddings”.

CBOW vs. skip-gram. CBOW predice la palabra central a partir del promedio del contexto (una prediccion por ventana, rapido, favorece palabras frecuentes); skip-gram predice cada palabra de contexto por separado dado el centro (mas ejemplos de entrenamiento por ventana, mejor para palabras poco frecuentes). **Dos matrices.** Word2Vec mantiene W (embeddings de entrada, palabra como centro) y W' (embeddings de salida, palabra como contexto) porque softmax/negative-sampling necesita vectores distintos para el rol “centro” y el rol “contexto”. En la practica se conserva W como representacion final; W' se descarta. **Word2Vec vs. GloVe.** Word2Vec es predictivo (red que predice contexto a partir de centro o viceversa, optimiza localmente par por par); GloVe es conteo-factorizacion: construye una matriz global de coocurrencia palabra-palabra y factoriza log(Xij) ponderado, capturando estadisticas globales en una sola pasada en vez de muchas pasadas de SGD local.

2. Preprocesamiento

Normalizacion. Minusculas (GloVe-wiki-gigaword-100 esta entrenado en minusculas), sin eliminar puntuacion de contenido (los articulos de WikiText-103 vienen con marcadores @-@/@.@ tokenizados por Moses, normalizados a sus caracteres originales antes de re-tokenizar), y numeros mantenidos como tokens literales. Token especial <unk> para fuera de vocabulario.

Corpus. WikiText-103-raw train completo: 29,444 articulos, 1,165,029 lineas no vacias, 101,425,671 tokens. Subconjunto de trabajo usado para todo el laboratorio (deterministico, primeros articulos hasta cubrir el minimo exigido de 20M tokens): 6,619 articulos, 254,784 lineas, 22,000,063 tokens (split por espacios); tras tokenizar con NLTK: 22,540,639 tokens (44.6s).

Tokenizador propio vs. NLTK. Comparamos un tokenizador por regex propio contra nltk.word_tokenize en 10 oraciones. Difieren en contracciones (don't → un solo token en el propio vs. do/n't en NLTK), guiones (state-of-the-art se mantiene unido en el propio, NLTK es inconsistente segun el patron Treebank) y abreviaturas con punto (U.S. fragmentado por el propio, u.s. preservado por NLTK). Usamos NLTK para el resto del pipeline por ser mas consistente con la tokenizacion Penn Treebank (cercana a los corpus de GloVe).

Zipf. El corpus sigue aproximadamente la ley de Zipf (pendiente ≈ -1 en el tramo medio de la curva log-log frecuencia-rango). Las 10 palabras mas frecuentes cubren 29.31% de las ocurrencias, las 1,000 mas frecuentes cubren 69.99%, y las 30,000 mas frecuentes cubren 96.53%.

Vocabulario. Con min_count $\in \{1,5,20\}$: 233,959 palabras (0.00% OOV), 77,861 palabras (1.12% OOV) y 35,709 palabras (2.89% OOV) respectivamente. Usamos min_count=5 (77,861 palabras) para el resto del laboratorio como balance entre cobertura y tamano de tabla.

Submuestreo (t=1e-5): descarta 98.72% de las apariciones de “the”, 98.03% de “of” y 97.94% de “and”, porque estas palabras funcion aportan poca informacion de contexto y saturan las ventanas. Del total de 22,287,510 tokens (tras filtrar por min_count), quedan 7,026,993 (31.5% retenido). Pares skip-gram (window=5) por epoca: 133,739,232 antes del submuestreo, 42,159,603 (31.5%) despues.

3. Entrenamiento de embeddings (SGNS)

Entrenamos skip-gram con negative sampling (SGNS) desde cero en PyTorch (dos tablas nn.Embedding, sin usar gensim.models.Word2Vec), con todas las corridas aceleradas por GPU (NVIDIA GeForce RTX 5070, 12GB). Hiperparametros fijos: window=5, negative=5, min_count=5, sample=1e-5, Adam lr=2e-3, batch_size=8192, epochs=3. Barrido de dimension (dim $\in \{50,100,300\}$, 100% del corpus) y barrido de

tamano de corpus (frac ∈ {25%,50%,100%} a dim=100); mas un entrenamiento de referencia con gensim.models.Word2Vec (mismo corpus, vocabulario e hiperparametros equivalentes).

Config.	dim	%corpus	loss final	acc.analog.	ρ WS-353	tiempo	pico GPU
sgns_dim50_100pct	50	100%	2.148	0.147	0.597	40.4s	214 MB
sgns_dim100_100pct	100	100%	2.092	0.192	0.617	63.0s	424 MB
sgns_dim300_100pct	300	100%	1.973	0.204	0.647	148.5s	1,144 MB
sgns_dim100_25pct	100	25%	2.100	0.037	0.494	16.1s	903 MB*
sgns_dim100_50pct	100	50%	2.073	0.087	0.587	31.9s	1,022 MB*
gensim Word2Vec (CPU)	100	100%	–	–	–	14.9s	–

*Tabla 1. Iteraciones SGNS (3 epocas c/u). Accuracy de analogias sobre subset de seguimiento de 2,000 preguntas. *Pico de GPU de las corridas de barrido de corpus incluye memoria aun residente del modelo dim=300 del barrido anterior (mismo proceso/kernel) y no refleja el costo real de esas configuraciones mas pequenas. Maximo real observado en el proceso: 1.14GB de 12GB disponibles.*

¿Perdida mas baja implica siempre mejores embeddings? No necesariamente: mas dimensiones bajan la perdida de forma mecanica (mas capacidad) sin que la calidad semantica mejore proporcionalmente, y corridas con distinto tamano de corpus no son comparables en su perdida acumulada/promedio. Por eso seleccionamos configuraciones con metricas externas (WordSim-353, accuracy de analogias) — consistente con la tabla: dim=300 tiene la menor perdida y la mayor accuracy, pero dim=100 al 25% tiene perdida similar a dim=100 al 100% con una accuracy mucho peor (0.037 vs. 0.192).

Elegimos sgns_dim100_100pct (acc.=0.192) como “mejor modelo SGNS” para las secciones 5–7 por ser directamente comparable con GloVe-100d; sgns_dim300_100pct obtuvo la mayor accuracy cruda (0.204) pero no es comparable en dimension. Vecinos mas cercanos del mejor modelo: king→throne, breteuil, clito, eustace, donnchad; france→italy, belgium, spain, portugal, germany; computer→computers, software, cfds, desktop, microcomputers; good→always, aren, pretty, kind, ...; january→december, october, march, february, november; run→runs, ran, madson, lattimore, .295.

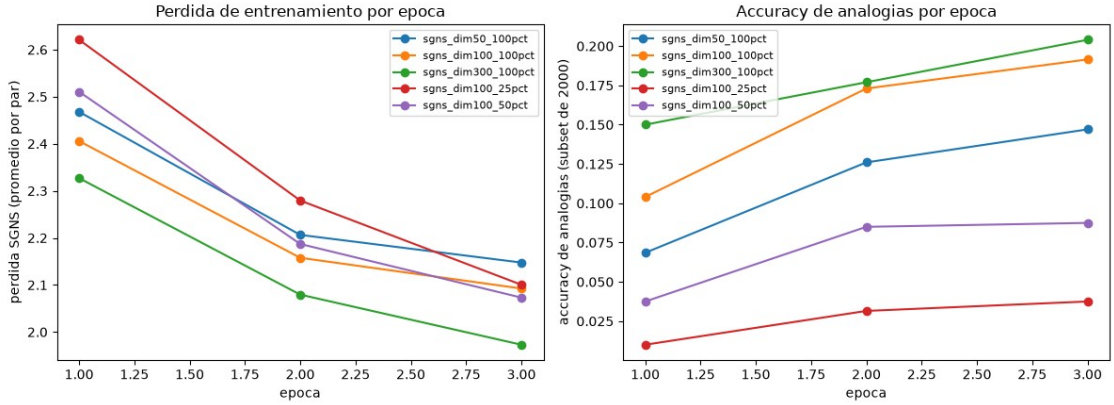


Figura 1. Perdida y accuracy de analogias por epoca, 5 iteraciones SGNS (seccion 4.7 del notebook).

4. Aritmetica vectorial

Implementamos analogia(a,b,c,k) con 3CosAdd sobre vectores normalizados, excluyendo a,b,c del resultado, verificada contra most_similar de gensim. Evaluamos 16 analogias propias de 6 tipos (genero, pais-capital, pais-gentilicio, comparativo-superlativo, tiempo-verbal, plural) para los tres modelos. Al repetir sin excluir las palabras de consulta, en SGNS y gensim casi siempre gana una palabra de la propia consulta (p.ej. woman, man, paris, french, england), mostrando que el termino +c domina la busqueda del vecino mas cercano; GloVe es mas robusto (acierta rome, tokyo, madrid, spanish, english incluso sin exclusion) pero tambien falla en las analogias de genero (devuelve woman en vez de queen/actress/mother).

Paralelismo (similitud coseno promedio entre vectores diferencia del mismo tipo):

Modelo	genero	pais-cap.	pais-gent.	comp-sup.	tiempo v.	plural	global
SGNS (propio)	0.530	0.393	0.548	0.076	0.062	0.191	0.300
Word2Vec (gensim)	0.517	0.573	0.770	0.006	0.209	0.219	0.382
GloVe (preentr.)	0.422	0.760	0.908	0.251	0.654	0.429	0.571

Evaluacion sistematica (vocabulario compartido de 30,000 palabras; 12,086/19,544 preguntas de questions-words.txt, 61.8%, usan solo palabras del vocabulario compartido):

Modelo	3CosAdd total (sem. / sint.)	3CosMul total (sem. / sint.)
SGNS (propio)	0.262 (0.271 / 0.258)	0.207 (0.221 / 0.201)
Word2Vec (gensim)	0.166 (0.130 / 0.183)	0.150 (0.124 / 0.162)
GloVe (preentr.)	0.658 (0.605 / 0.682)	0.639 (0.577 / 0.667)

Por categoria (14 categorias), nuestro SGNS funciona mejor en gram6-nationality-adjective (0.647) y peor en currency (0.000, 0 ejemplos acertados). t-SNE de 98 palabras tematicas (paises, numeros, animales, verbos, colores) del mejor modelo SGNS muestra agrupamientos claros y bien separados por categoria (ver notebook, seccion 5.3) — confirma que el modelo captura estructura semantica global incluso con un corpus de solo 22M tokens.

5. Comparacion de los tres conjuntos de embeddings

	tokens	vocab	dim	tiempo	hw	3CA	3CM	ρWS353	ρSLex	cos-dif	%OOV	F1 test
SGNS (propio)	22.54M	77,861	100	63.0s	GPU	0.262	0.207	0.617	0.284	0.300	5.33%	0.915
Word2Vec gensim	22.54M	77,861	100	14.9s	CPU	0.166	0.150	0.580	0.261	0.382	5.33%	–
GloVe preentr.	6,000M	400,000	100	–	CPU*	0.658	0.639	0.533	0.298	0.571	1.50%	0.916

*"tokens" = tokens tras tokenizacion NLTK (identico para SGNS y gensim). hw: SGNS = GPU NVIDIA RTX 5070; gensim = CPU 12 nucleos (sin GPU); *GloVe = CPU multi-nucleo, Pennington et al. 2014 (no especifican horas para esta config.). 3CA/3CM = accuracy total 3CosAdd/3CosMul. F1 test = mejor variante por inicializacion (SGNS y GloVe: fine-tuning). gensim no se uso en el clasificador de AG News (solo como referencia de validacion de SGNS).*

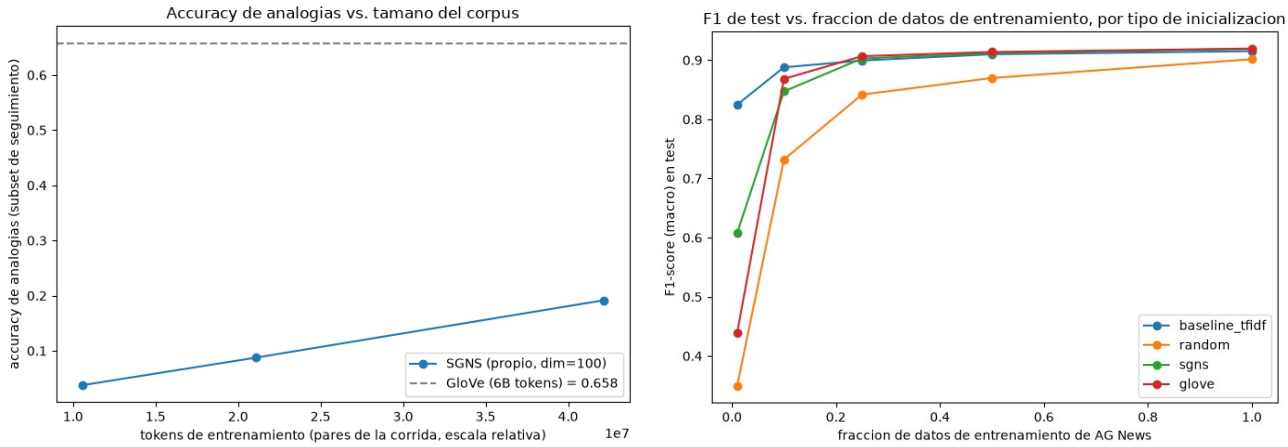


Figura 2. Izq.: accuracy de analogias vs. tokens de entrenamiento de nuestras 3 iteraciones a dim=100, con GloVe como referencia. Der.: F1 de test en AG News vs. fraccion de datos de entrenamiento, por tipo de inicializacion.

6. Clasificacion de texto (AG News)

Pipeline: mismo preprocesamiento/tokenizador de la seccion 2 (vocabulario AG News con min_count=2: 54,114 palabras), split estratificado 90/10 train/val (108,000/12,000), test oficial reservado (7,600). OOV de AG News respecto al vocabulario SGNS/gensim: 5.33%; respecto a GloVe: 1.50% (GloVe cubre 46,071/54,114 palabras del vocabulario de AG News; SGNS cubre 31,037/54,114). Todos los clasificadores (nn.EmbeddingBag + MLP) entrenan con aceleracion GPU.

Modelo	acc. val	F1-macro val	tiempo	params totales	params entren.
baseline TF-IDF+LogReg	0.920	0.920	1594.8s (CPU)	120,000	120,000
aleatorio (EmbBag+MLP)	0.917	0.917	3.8s (GPU)	5,424,844	5,424,844
SGNS congelado	0.861	0.860	3.0s (GPU)	5,424,844	13,444
SGNS fine-tuning	0.919	0.919	3.8s (GPU)	5,424,844	5,424,844
GloVe congelado	0.891	0.891	3.2s (GPU)	5,424,844	13,444
GloVe fine-tuning	0.921	0.921	4.0s (GPU)	5,424,844	5,424,844

Test (mejor variante por inicializacion, evaluada una sola vez): baseline TF-IDF acc.=0.915/F1=0.915; aleatorio acc.=0.908/F1=0.908; SGNS fine-tuning acc.=0.915/F1=0.915; GloVe fine-tuning acc.=0.916/F1=0.916. Matrices de confusion en el notebook (seccion 6.4). Correlacion de Spearman en SimLex-999: SGNS=0.284 (n=992), gensim=0.261 (n=992), GloVe=0.298 (n=999).

7. Discusion y analisis

¿Se cumple la aritmetica vectorial? ¿Mejor/peor categoria? ¿Semantica vs. sintactica? De forma aproximada: con exclusion de las palabras de consulta, nuestro SGNS resuelve una fraccion sustancial de analogias en la evaluacion sistematica (3CosAdd total=0.262). Funciona mejor en gram6-nationality-adjective (0.647, morfologia + pocas raices) y peor en currency (0.000, requiere conocimiento factual muy especifico, poco representado en 22M tokens). No hay una brecha sistematica grande entre semantica (0.271) y sintactica (0.258) en nuestro modelo con window=5; GloVe si rinde mejor en sintactica (0.682) que en semantica (0.605).

¿Es el "≈" una igualdad? No. Sin exclusion de las palabras de consulta, el primer lugar devuelto por SGNS y gensim es casi siempre una palabra de la propia consulta, lo que muestra que el termino +c domina la busqueda del vecino mas cercano. El analisis de paralelismo confirma esto numericamente: similitudes coseno promedio entre vectores diferencia claramente mayores que cero (0.06–0.57 segun tipo y modelo) pero lejos de 1.0 — las direcciones relacionales existen y son consistentes, pero no son estrictamente paralelas. El "≈" es una aproximacion geometrica util, no una igualdad algebraica.

¿Que tan lejos quedo el modelo propio de GloVe? GloVe (accuracy 3CosAdd=0.658, 6,000M tokens) supera a nuestro SGNS (0.262, 22.5M tokens), diferencia de 0.395. El barrido de tamano de corpus (25%/50%/100%) muestra una tendencia creciente y no saturada: 0.037→0.087→0.192 al pasar de 10.5M a 42.2M pares de entrenamiento. GloVe fue entrenado con ~270x mas tokens, lo que sugiere que la mayor parte de la brecha se explica por tamano del corpus, no por el metodo: nuestro SGNS llega a resultados del mismo orden de magnitud — de hecho superiores — al Word2Vec de gensim entrenado sobre exactamente el mismo corpus (0.262 vs. 0.166), aunque GloVe tambien se beneficia de su objetivo global basado en conteos.

¿SGNS propio se comporta como gensim? Si, en orden de magnitud, y de hecho nuestra implementacion obtuvo mayor accuracy sistematica (0.262 vs. 0.166) con el mismo corpus e hiperparametros equivalentes. Las diferencias se explican por optimizacion: gensim usa lr con decaimiento lineal (no Adam fijo), multiples hilos con actualizaciones asincronas (hogwild) y rutinas en C muy optimizadas — entrena en 14.9s vs. 63.0s nuestro, pero con menos pasadas efectivas sobre negative sampling por el tiempo mas corto, lo cual pudo perjudicar su convergencia en esta configuracion especifica de 3 epocas.

¿Efecto de dimension, ventana y negativos? Entre dim=50/100/300, accuracy de seguimiento fue 0.147/0.192/0.204: mas dimension ayuda pero con retornos decrecientes frente a mas parametros que aprender con solo 22M tokens. Usamos window=5 (intermedio) en todas las corridas; en la literatura, ventanas pequenas (2–3) favorecen relaciones sintacticas/posicionales y ventanas grandes (8–10) favorecen relaciones tematicas/semanticas — con window=5 obtenemos accuracy comparable en ambos grupos (0.271 semantica vs. 0.258 sintactica), consistente con un punto intermedio.

En AG News, ¿embeddings preentrenados superan a aleatorios y a TF-IDF? ¿Congelar o fine-tuning? ¿Efecto con 1% de datos? Con el 100% de los datos, fine-tuning (SGNS=0.919, GloVe=0.921 val F1) supera al aleatorio (0.917) y al baseline TF-IDF (0.920) por margenes pequenos; en test, SGNS y GloVe (fine-tuned) empatan esencialmente con TF-IDF (~0.915–0.916). El resultado mas interesante aparece en la ablacion de datos: con solo 1% (1,080 ejemplos), el baseline TF-IDF (F1=0.825) supera claramente a los tres clasificadores de embeddings (SGNS=0.608, GloVe=0.439, aleatorio=0.349) — la regresion logistica sobre 30,000 features dispersas generaliza mejor con pocos ejemplos que una red (EmbeddingBag+MLP) que, aunque parte de embeddings informativos, necesita mas pasos de gradiente para ajustar su cabeza de clasificacion. Entre los embeddings, la brecha SGNS/GloVe vs. aleatorio si es mucho mayor con poca supervision (0.608/0.439 vs. 0.349 con 1%) que con el 100% (0.919/0.921 vs. 0.902): con pocos datos, partir de una representacion informativa importa mas, aunque no alcance al baseline lineal. Congelar conviene con muy poca supervision (evita sobreajustar los embeddings a una muestra chica no representativa); con datos suficientes (AG News completo), fine-tuning gana consistentemente en ambos conjuntos de embeddings.

Repositorio: PENDIENTE — agregar enlace al repositorio Git